Timeline (Shop Dashboard) — Handoff Notes

Prepared by: Peter Skvarla · Twoseven Inc.**Current version:** REV50**Live at:** https://peterskvarla-sys.github.io/shop-timeline/

---

1. What this thing actually is

A Gantt-style production scheduling tool for the shop. Three facts explain the whole architecture:

1. **It is one HTML file.** No server, no build step, no framework. All the code — layout, styling, logic — lives in a single index.html. You edit that file, you upload that file, you're done.
2. **GitHub Pages serves it.** GitHub hosts the file at a public web address for free.
3. **SharePoint stores the data.** Projects, tasks, and staff live in SharePoint lists. The app reads and writes them through Microsoft Graph.

Staff open it as a tab in Microsoft Teams. They sign in with their normal Microsoft account — no new passwords.

---

2. The credentials and addresses you'll need

What	Value
Live URL	https://peterskvarla-sys.github.io/shop-timeline/
GitHub repo	GitHub - peterskvarla-sys/shop-timeline (branch main, root folder)
Entra app name	shop-timeline-app
Application (client) ID	5ba3aabe-81f7-41c9-92a4-83a45d5407ab
Directory (tenant) ID	70aa5330-416f-48cb-a64f-1a89f0196577
SharePoint site	https://twosevennet.sharepoint.com/sites/TWOSEVENINC
SharePoint lists	ShopTimeline_Projects, ShopTimeline_Tasks, ShopTimeline_Staff

Those two IDs are not secrets. They're meant to be public and sit in plain sight in the HTML. There is no client secret and no API key anywhere in this project — by design. The security comes from Microsoft login, not from hiding a string.

---

3. How the login plumbing works (plain English)

The chain is four links long:

1. A user opens the page. A Microsoft library called **MSAL** pops up a Microsoft sign-in window.
2. Microsoft checks the user against **Twoseven's Entra directory** (the company's list of employees), and checks that our app is allowed to ask.
3. Microsoft hands back a temporary **access token** — a signed permission slip, good for about an hour.
4. The app attaches that token to every request it makes to SharePoint. SharePoint sees a valid token and answers.

The app can only do what the token permits. We asked for exactly two permissions and nothing more.

---

4. Setup steps that were performed (in order)

This is the part that was hard to see happening. Here's what was actually done, and why.

Step 1 — Getting permission to create an app at all

Kenny granted the **Application Developer** role in Entra. Without it you can't register apps. *This is a Kenny task if it ever needs redoing.*

Step 2 — Register the app

Entra admin center (entra.microsoft.com) → **Entra ID → App registrations → New registration**.

- Named shop-timeline-app
- **Single tenant** ("My organization only") — internal staff only, no outside accounts
- Microsoft returned the client ID and tenant ID listed above

Step 3 — Grant API permissions

API permissions → Add a permission → Microsoft Graph → Delegated permissions, then added:

Permission	What it allows
User.Read	Read the signed-in user's own profile
Sites.ReadWrite.All	Read and write SharePoint list data

"Delegated" is the important word — the app acts *as the signed-in person*, with their access, never as an all-powerful service account.

Step 4 — Admin consent (the step that got stuck)

Sites.ReadWrite.All requires an administrator to approve it once for the whole company. The **Grant admin consent** button was greyed out for Peter's role. **Kenny clicked it.** Login was blocked until he did.

If the app ever stops authenticating for everyone at once, suspect this.

Step 5 — Register the redirect URI

Authentication → Add a platform → Single-page application (SPA). Two URIs are registered:

- http://localhost:3000 — for local testing
- The GitHub Pages URL — for production

Three details that matter:

- It must be registered as **SPA**, not "Web." SPA uses the modern auth-code-with-PKCE flow, which is why no client secret exists.
- The **"Access tokens" and "ID tokens" checkboxes are deliberately left unchecked.** Those are for a legacy flow. Ticking them triggers security warnings and buys nothing.
- **The URL never changes on deploys.** Swapping the file doesn't require touching this panel. Only a change of hosting address would.

Step 6 — Build the SharePoint lists

Created on the **Twoseven Inc. Team Site** — deliberately *not* a personal OneDrive, so the whole team can reach the data. Two conventions were locked in early:

- **Column names contain no spaces** (jobCode, not Job Code). SharePoint silently mangles spaces into _x0020_ in the internal name, which then breaks the API calls.
- **Arrays are stored as JSON text.** SharePoint columns are flat, so things like activeDepartments and ticketNodes get packed into a text column as JSON and unpacked by the code.
- **appId column on every list.** SharePoint assigns its own row IDs, but the app has its own IDs and links records by them. appId preserves those links.

The ShopTimeline_ prefix keeps our data visibly separate from the site's pre-existing company lists. **Do not touch or reuse those.**

Step 7 — Hosting (the detour worth knowing about)

Hosting the file directly on SharePoint was tried first **and it failed.** On a modern Teams-connected site, SharePoint insisted on *downloading* the HTML file rather than displaying it — even after Custom Scripts were enabled. This is a platform behavior, not a fixable bug. Don't spend a day re-litigating it.

GitHub Pages was the fix and has worked reliably since.

Step 8 — Pin it in Teams

Added as a Website tab in the **Production Schedule** channel, pointed at the GitHub Pages URL. Note: in new Teams, Website tabs open in Edge rather than embedding inline. That's expected behavior, not a misconfiguration.

---

5. How to ship an update

1. Rename the new version file to **index.html** (GitHub Pages only serves that name at the root).
2. Go to GitHub - peterskvarla-sys/shop-timeline.
3. **Add file → Upload files**, drag it in, commit to main. This overwrites the live version.
4. Wait 30–60 seconds for the rebuild.
5. Hard-refresh (Ctrl+Shift+R).

No Entra changes. No re-pinning the Teams tab. GitHub keeps every previous version, so rollback is a couple of clicks.

One file must never be deleted from the repo: msal-browser.min.js. It's the Microsoft login library, stored locally rather than loaded from a CDN. It sits next to index.html. Remove it and login dies instantly.

---

6. Traps that have already cost time

The redirect URI bug. Using window.location.origin for redirectUri causes the browser to *download a file* after login instead of returning to the app. The correct form, already in the code, is:

window.location.href.split('#')[0].split('?')[0]

If post-login downloads ever reappear, look here first.

Stale versions in Teams. The tab sometimes shows an old REV after a successful deploy. Diagnose it this way: open the raw github.io URL in a **private Edge window** and check the REV number in the top-left corner.

- Old REV there too → the deploy didn't land (wrong branch, or the file didn't overwrite index.html at root).
- New REV there but old REV in Teams → it's Teams' webview cache. Remove and re-add the tab.

Verify schema in the code before declaring a column missing. This burned multiple sessions: the label column on ShopTimeline_Tasks was repeatedly reported as missing when it had existed the whole time. Search the HTML for the field name before asking Kenny for anything.

Test both pages, always. The new-project draft page (#/project/new) and the saved-project page behave differently. A fix that works on one frequently doesn't work on the other.

---

7. When you need Kenny

Kenny Ito, IT/SharePoint admin. Required for:

- Any new SharePoint column or list
- Anything tenant-level in Entra (admin consent, role grants)
- The pending GitHub org transfer

Everything else — code, deploys, UI — needs no one.

---

8. Open items

Item	Status
2FA on the GitHub account	**Not enabled. Highest-priority security gap** — this personal account hosts a production company tool.
Transfer repo to a Twoseven org account	Waiting on Kenny to create the org
ShopTimeline_Tasks2 list (to-do tasks)	Designed, not yet created in SharePoint. App degrades gracefully without it — keeps tasks in session and shows one warning.
Dash view	Designed, not built

---

9. Development workflow

The app carries a regression test suite — **276 assertions across 6 jsdom suites** — run before every release.

Standing conventions in the code:

- Subtasks move with their department bar by default; a **Link toggle** disables it
- Right-click is the primary creation verb throughout the chart
- Events render as yellow diamonds with black outlines
- Every created item gets a default date
- Every mutating action shows an **Undo button** in its toast

Patching a file this large is done with Python string-replacement scripts guarded by assert s.count(anchor) == 1, so an ambiguous anchor fails loudly instead of patching the wrong function. Output is syntax-checked with Node before shipping.

Keep this document current — it's the continuity artifact. Hand it over alongside the latest HTML file and the test suite.